DDCD Project Proposal

(The Dungeons & Dragons Character Database Project)

Skylar Magouliotis

GitHub: skycreated

Team Lead

Spencer Mulder

GitHub: Runboy07

Architect

Kyle Yee

GitHub: kyleyee-dev

Developer



Intermediate Programming 2
COSC 183 – T05
Ada Ajunwa

Project Description

For this project, our team has created a client and server program that allows multiple users to perform CRUD (create, read, update, delete) operations on a database. The project idea is inspired by the Character Sheets from the TTRPG Dungeons and Dragons (5th edition) and will allow the user to access information quickly and accurately.

Project Overview

I. What Are Characters and Character Sheets?

In the Dungeons and Dragons game, each player creates a character (also called an adventurer) and teams up with other adventurers (played by friends).

Character sheets are a concise arrangement of statistics and character details that comply with the rules and standards of the game that they are associated with (in this case, Dungeons & Dragons 5th Edition). Character sheets provide the player or dungeon master (DM; the person who runs the game) with crucial and necessary numerical and standardized information for player characters that allow the DM or player to utilize this information accurately and quickly.

II. Why Do We Use Character Sheets in TTRPG's?

A TTRPG is a table-top role-playing game.

Character Sheets allow complex information to be concentrated into an easy-to-reference, clear, and concise format that is more efficient (and significantly quicker) for the player or DM to use. Normally a player or DM would have to create and reference this information with a physical book or paper, so creating this program has made a more efficient way to create, reference, and filter essential character information.

Character Sheets provide elaborate information and statistics about a character that contribute to an exciting and "realistic" experience for the player.

Table and Data Descriptions

I. What does the database hold?

The "character" database holds all the essential information to create, update, read, or delete a character. It contains a "character table" that has all the important fields to store a player character.

II. What is the name of the table, and what data does it hold?

The "character table" holds all the essential data that makes up a character. It consists of physical characteristics of that character, main statistics for all their abilities, and their non-physical characteristics like background and alignment.

III. What are the field names and data descriptions for those fields?

Table Field	**"character table" Data Description**	**Data Details**
CharacterID	Identification Number of the character	Primary Key – int
Name	Character Name	varchar(45) – string
Class	Character type (the main fighting style and how the character works – can be one of the following options: Barbarian, Bard, Cleric, Druid, Fighter, Monk, Paladin, Ranger, Rogue, Sorcerer, Warlock, Wizard)	varchar(45) – string
Race	Character race (the species of the character – can be one of the following options: Half-Orc, Human, Dwarf, Wood Elf, Halfling, Half-Elf, Dragonborn, Gnome, Tiefling, Dark Elf, High Elf)	varchar(45) – string
Level	Character level (minimum level is 1 and maximum level is 20)	int
Alignment	Character demeanor (how the character acts – can be one of the following options: lawful good, neutral good, chaotic good, lawful neutral, true neutral, chaotic neutral, lawful evil, neutral evil, chaotic evil)	varchar(45) – string
Background	Character backstory (a character's personal history, motivations, and defining traits – can be one of the following options: Outlander, Entertainer, Acolyte, Hermit, Soldier, Noble, Charlatan, Sage)	varchar(45) – string
AC	Character Armour Class (numerical value that measures defensive capability in combat)	int
HP	Character Hit Points (numerical value that measures health and vitality)	int
Speed	Character speed (how many feet the character can move per turn)	int
Age	Character age (how old a character is in years)	int
Weight	Character weight (how much a character weighs in pounds)	int
Height	Character height (how tall a character is in feet)	int
Strength	Character strength (a numerical value ability score that determines the strength of a character)	int
Dexterity	Character dexterity (a numerical value ability score that determines how dextrous a character is in reference to agility, reflexes, and balance)	int
Constitution	Character constitution (a numerical value ability score that measures health, stamina, and physical resilience)	int
Intelligence	Character intelligence (a numerical value ability score that measures mental sharpness, analytical ability, and memory)	int
Wisdom	Character wisdom (a numerical value ability score that measures perception, intuition, and practical knowledge)	int
Charisma	Character charisma (a numerical value ability score that measures personality, charm, and social skills)	int

Process and Functionalities Between the Client, Server, and Database

I. Important Details

Our project contains a Main, Server, Client, Database, and CreateCharacter class.

The Main class confirms if a server is selected and then creates a new instance of the server object and runs that. If it is not confirmed that a server has been selected, it will create a new instance of client object and runs that instead. A new instance of the database object is created and selects the associated information to be retrieved and displayed from the database.

When the program is run, the menu prompts the user to confirm if it is a server or a client. When the server starts, it waits for the client on that port number. While the server is running, it waits for the client to connect. Multiple clients can access the information concurrently. The server releases a thread from the thread pool that will do the specific task for the client (as the client requests). The server collects the data that the client is sending and accepts the input and converts the bytes to string (via input stream reader) and then makes sure that there is synchronization (via buffered reader). The data is then pushed back to the client, we instantiate a message coming from the client, and while the data is not null, we "talk" to the database for information before sending it back to the client.

The client requires the port number and IP address. We use the local host to simulate communication on the same system (so that the server is on the same system as the client). In other cases, where the server is on a different system, we would need that IP address instead of using the local host. When the socket is created, it checks that there is data present and runs while the data is not empty. A response from the server is displayed to confirm the input.

The database requires the url, username, and password to operate. To connect, it required the jdbc, database type (mysql), IP address (in this case it is the local host because we are using our local system), port number, and database name ("character"). With statements, we create a statement object to "open the door" to the database, and then we insert the query using the execute function, and then the result is processed. Connections are closed afterwards.

II. Manipulation of Returned Data

On the client's request to create, the CreateCharacter class takes all the required variables that align with the values in the table from the database and creates a random character. That randomly generated character is then loaded into the database and displayed to the user. The user will get an array of the values, as well as a short character summary that they can read.

On the client's request to read data, they must provide the ID of the character that they wish to view, or they can choose to view all the characters currently in the database. If there are records present, they will be displayed in an array. If they are just interested in viewing a single character, they will receive a character summary as well.

On the client's request to update data, they must provide the ID of the character they wish to update, and then they will be prompted with a menu to determine what field they wish to change. They can then adjust a character within the formal parameters (for example, the ID number should never be changed, and the level maximum is 20).

On the client's request to delete data, they must provide the ID of the character they wish to delete.

We use prepared statements within the database class to execute the CRUD (create, read, update, delete) operations and we have appropriate return messages where applicable (such as "no character record found").

Conclusion

This project facilitates our ability to understand client/server relationships and how we can manipulate a database by utilizing CRUD (create, read, update, delete) operations.

Please note that this proposal will be submitted prior to the due date of the project, so some features may be changed, updated, or expanded and enhanced.

This project meets all of the requirements set forth in the requirements document, which includes the following: A text based menu; CRUD (create, update, read, delete) operations; a server class; a client class; a database class; provides values to be inserted; uses prepared statements for operations; server is multithreaded; server acts as a proxy to accept client requests to query the database; server accepts parameters from the client and constructs a query which it sends to the database; server receives results back from the database server and forwards the results to the client; receives requested data or nothing; data is displayed in a message; returned messages are appropriate (if there is no data or wrong information, display message describes this); returned data is creatively manipulated and displayed to the user.

By doing this project, we have developed a better understanding of multi-threading, network programming, and databases.

References

Wizards of the Coast LLC. (2014). Players Handbook. In J. Wyatt, R. J. Schwalb, & B. R. Cordell, *Players Handbook.* Renton, USA: Wizards of the Coast LLC. Retrieved May 24, 2026